

AI 视觉对话助手

设计文档

用户故事与成本控制策略

版本：1.0

日期：2026-06-12

目录

- 1. 项目概述
- 2. 用户故事
 - 2.1 计划实现的用户故事
 - 2.2 最终实现状态
- 3. 成本控制策略
 - 3.1 考虑到的成本控制技巧
 - 3.2 实际采用的策略
- 4. 技术架构
- 5. 端云协同设计
- 6. 总结与展望

1. 项目概述

AI 视觉对话助手是一款基于多模态大模型技术的智能交互应用，旨在让AI能够"看到"用户摄像头捕捉的实时画面，"听到"用户通过麦克风说出的话语，并给予恰当、自然、有针对性的回应。该应用融合了计算机视觉、语音识别、自然语言处理和语音合成等多项AI能力，构建了一个完整的端到端多模态交互体验。

在技术实现层面，本应用采用了端云协同的架构设计。客户端（浏览器端）负责摄像头和麦克风的媒体采集、语音活动检测（VAD）、智能帧采样、图像压缩以及场景变化检测等计算任务，最大限度地减少不必要的云端API调用；云端则负责视觉理解（VLM）、语音识别（ASR）、对话生成（LLM）和语音合成（TTS）等需要大模型算力的核心任务。这种分工方式既保证了交互的实时性和准确性，又有效控制了运营成本。

本设计文档将详细阐述项目的用户故事规划与实现状态，以及成本控制策略的思考与实践，为后续的产品迭代和运营优化提供参考依据。

2. 用户故事

2.1 计划实现的用户故事

在项目规划阶段，我们基于多模态交互的核心场景，梳理了以下用户故事。这些故事覆盖了从基础的音视频采集到高级的智能交互体验，体现了产品从MVP到完整功能的演进路径。每个用户故事都从用户视角出发，描述了期望的功能和交互体验。

编号	名称	描述	优先级
US-01	开启摄像头	作为用户，我希望一键开启摄像头，让AI能够看到我面前的画面，以便获得基于视觉内容的智能回应	高
US-02	开启麦克风	作为用户，我希望一键开启麦克风，通过语音与AI对话，无需手动输入文字	高
US-03	视觉问答	作为用户，我希望询问AI关于摄像头画面中的内容（如"我手里拿的是什么？"），AI能准确识别并回答	高
US-04	语音识别	作为用户，我希望说出的话能被准确转写为文字，并在对话界面中显示	高
US-05	语音回复	作为用户，我希望AI的回复能通过语音朗读出来，实现真正的语音对话体验	高
US-06	文字输入	作为用户，我希望在不方便说话时，也能通过文字输入与AI交流	中
US-07	对话历史	作为用户，我希望看到完整的对话历史记录，方便回顾之前的交流内容	中
US-08	场景变化感知	作为用户，我希望当画面发生显著变化时，AI能主动注意到并提醒我	中

编号	名称	描述	优先级
US-09	成本统计	作为用户/运营者，我希望看到API调用的统计数据，了解各功能的成本消耗	中
US-10	手动拍照提问	作为用户，我希望手动点击按钮拍摄当前画面，并针对该画面向AI提问	中
US-11	多语言支持	作为用户，我希望AI能支持中英文双语交互	低
US-12	对话导出	作为用户，我希望能将对话记录导出保存	低

2.2 最终实现状态

经过开发实现，以下是各用户故事的最终完成状态。高优先级的用户故事全部实现，中优先级的核心功能也已落地，部分低优先级功能留待后续迭代。

编号	名称	状态	说明
US-01	开启摄像头	已实现	支持一键开关摄像头，实时视频预览显示在界面中
US-02	开启麦克风	已实现	支持一键开关麦克风，集成VAD语音活动检测，仅在说话时录音
US-03	视觉问答	已实现	通过VLM多模态大模型实现视觉理解，支持针对画面内容提问
US-04	语音识别	已实现	集成ASR语音识别，支持中文语音转文字，录音结束后自动识别
US-05	语音回复	已实现	集成TTS语音合成，AI回复自动朗读，支持开关控制
US-06	文字输入	已实现	提供文字输入框，支持键盘输入与语音输入两种方式
US-07	对话历史	已实现	完整的对话历史显示，区分用户消息和AI回复，标注视觉/文字模式
US-08	场景变化感知	已实现	客户端帧差异检测，画面变化超过阈值时自动携带图像发送给AI
US-09	成本统计	已实现	实时统计视觉/文字/ASR/TTS调用次数、帧发送/跳过数等指标
US-10	手动拍照提问	已实现	提供"拍照提问"按钮，手动捕获当前帧并发送给AI
US-11	多语言支持	部分实现	当前支持中文交互，底层模型具备英文能力，待后续优化提示词
US-12	对话导出	未实现	留待后续迭代，可扩展为导出Markdown或PDF格式

总体实现率为 10/12（83%），其中高优先级功能 100% 实现，中优先级功能 100% 实现，低优先级功能实现 50%。未实现的功能（对话导出）属于增值特性，不影响核心交互体验的完整性。

3. 成本控制策略

3.1 考虑到的成本控制技巧

在多模态AI应用的运营中，API调用成本是最主要的可变支出。视觉理解（VLM）的调用成本远高于纯文字对话（LLM），语音识别（ASR）和语音合成（TTS）也会产生持续的费用。如果不加控制地频繁调用这些API，运营成本将快速攀升。因此，我们在项目规划阶段就系统地思考了以下成本控制技巧，涵盖了从数据采集、传输、处理到存储的全链路优化。

编号	策略名称	描述	优先级
CC-01	端侧VAD语音活动检测	在客户端实现基于音量阈值的VAD，仅在检测到用户说话时才启动录音和ASR调用，避免静默段的无效API消耗	高
CC-02	智能帧采样策略	非对话状态下每5秒采样一次画面，对话状态下每2秒采样一次，避免每帧都发送图像	高
CC-03	图像压缩与缩放	发送前将图像缩放至512px宽度，JPEG质量压缩至60%，大幅减少图像token消耗	高
CC-04	场景变化检测	对比前后帧的像素差异，画面无显著变化时不重复发送图像，减少冗余VLM调用	高
CC-05	分级模型调用	纯文字对话使用轻量LLM，仅在涉及视觉内容时才调用VLM多模态模型，降低单次调用成本	高
CC-06	对话历史压缩	当对话历史超过一定长度时，对早期消息进行摘要压缩，减少输入token数量	中
CC-07	客户端缓存	对相同或相似的问题缓存AI回复，避免重复调用	中
CC-08	请求节流	限制单位时间内的API请求频率，防止异常或滥用导致成本飙升	中
CC-09	音频格式优化	录音使用较低采样率和压缩格式，减少ASR传输数据量	中
CC-10	TTS文本分段	对长文本进行智能分段，仅合成用户需要的部分，避免全文合成	低
CC-11	WebSocket长连接	使用WebSocket替代HTTP短连接，减少连接建立开销	低
CC-12	模型量化/蒸馏	对频繁调用的模型进行量化或使用蒸馏版本，降低推理成本	低

3.2 实际采用的策略

经过技术可行性和成本收益的权衡，我们最终采用了以下策略。选择的标准是：实现复杂度适中、成本节省效果显著、不影响用户体验。以下是每项策略的详细说明和预期效果。

编号	策略	状态	详细说明与效果预估
CC-01	端侧VAD	已采用	基于Web Audio API的AnalyserNode实现实时音量监测，设定音量阈值（可调节），仅在音量超过阈值时启动MediaRecorder录音。静默段完全不触发ASR调用，预计节省60-80%的ASR调用量
CC-02	智能帧采样	已采用	非对话状态5秒/次，对话状态2秒/次。相比每秒采样的方案，预计减少60-80%的帧发送量。采样间隔可根据网络状况动态调整
CC-03	图像压缩缩放	已采用	Canvas缩放至512px宽度，JPEG quality=0.6。原图1920x1080约500KB，压缩后约30-50KB，减少90%的传输数据量和VLM图像token消耗
CC-04	场景变化检测	已采用	对比前后帧的Canvas像素数据，计算差异比例。差异低于5%时跳过发送，避免画面静止时的冗余VLM调用。预计在办公等静态场景下节省50%以上的VLM调用
CC-05	分级模型调用	已采用	hasImage=true时调用VLM（createVision），hasImage=false时调用LLM（create）。VLM单次调用成本约为LLM的3-5倍，通过分级调用可将平均单次成本降低40-60%
CC-06	对话历史压缩	已采用	客户端维护最近10条对话历史，超出部分不发送。避免长对话导致输入token线性增长，预计节省30-50%的输入token
CC-08	请求节流	已采用	前端设置isLoading状态锁，AI回复期间禁止重复发送请求。防止用户快速点击导致的重复调用
CC-09	音频格式优化	已采用	使用webm/opus格式录音，采样率16kHz，兼顾识别准确率和文件大小。相比WAV格式，文件大小减少约80%

未采用的策略及原因：

编号	策略	未采用原因
CC-07	客户端缓存	对话场景高度个性化，缓存命中率低，实现复杂度与收益不匹配
CC-10	TTS文本分段	当前TTS输入限制1024字符，应用场景下回复长度通常在限制内，无需分段
CC-11	WebSocket长连接	当前交互频率不高，HTTP短连接开销可接受，后续高并发场景可升级
CC-12	模型量化/蒸馏	属于模型侧优化，应用层无法控制，需依赖模型提供商的成本优化

综合来看，已采用的8项成本控制策略覆盖了数据采集、传输、处理三个关键环节，形成了"减少无效调用、降低单次成本、控制数据规模"的三位一体成本控制体系。在典型

使用场景下，预计可降低60-75%的API调用成本，同时保持良好的用户体验。

4. 技术架构

本应用采用前后端分离的架构设计，前端基于Next.js 15框架构建，后端使用Next.js API Routes实现。所有AI能力通过z-ai-web-dev-sdk统一调用，确保了接口的一致性和可维护性。以下表格展示了各模块的技术选型和职责划分。

模块	技术选型	职责
前端-摄像头	getUserMedia API + Canvas	视频采集、帧捕获、图像压缩、场景变化检测
前端-麦克风	Web Audio API + MediaRecorder	音频采集、VAD语音活动检测、录音控制
前端-对话UI	React + Tailwind CSS + shadcn/ui	对话界面、消息展示、控制面板、成本统计
后端-视觉对话	z-ai-web-dev-sdk VLM/LLM	多模态视觉理解 (createVision) 和纯文字对话 (create)
后端-语音识别	z-ai-web-dev-sdk ASR	音频base64转文字，支持中文语音识别
后端-语音合成	z-ai-web-dev-sdk TTS	文字转MP3音频，支持多种音色和语速调节

架构设计的核心原则是"端侧做减法，云端做加法"。端侧尽可能多地承担数据预处理和过滤工作，只将真正需要AI处理的数据发送到云端，既降低了延迟，又控制了成本。云端专注于核心的AI推理任务，通过SDK的单例模式复用连接，减少初始化开销。

5. 端云协同设计

端云协同是本应用成本控制的核心设计理念。通过合理分配计算任务，在端侧完成数据过滤和预处理，在云端执行AI推理，实现了性能与成本的最优平衡。以下详细描述了各关键环节的端云分工策略。

5.1 视觉数据流

摄像头采集的原始视频流（通常30fps）首先在端侧经过帧采样模块，按照当前状态（对话/非对话）选择不同的采样间隔。采样得到的帧经过图像压缩模块缩放至512px宽度并压缩JPEG质量。随后，场景变化检测模块对比前后帧的像素差异，仅在画面发生显著变化时才将图像发送至云端VLM进行视觉理解。这一流程将原始30fps的视频流压缩为0.2-0.5fps的有效图像发送频率，大幅降低了VLM的调用次数和图像token消耗。

5.2 语音数据流

麦克风采集的音频流首先经过端侧VAD模块的实时监测。VAD基于Web Audio API的AnalyserNode计算实时音量，当音量超过可调节的阈值时启动MediaRecorder录音，当音量持续低于阈值一段时间后停止录音。录音数据以webm/opus格式编码，兼顾文件大小和识别准确率。录音结束后，音频数据以base64编码发送至云端ASR进行语音识别。这一机制确保了只有在用户真正说话时才产生ASR调用成本，静默段完全不消耗API资源。

5.3 对话数据流

用户的输入（语音转写文字或直接文字输入）与可选的图像数据一起发送至云端对话接口。云端根据是否携带图像自动选择调用VLM（createVision）或LLM（create），实现分级模型调用。对话历史在客户端维护，仅发送最近10条消息，避免长对话导致的输入token膨胀。AI的回复文字通过TTS合成为语音返回客户端播放，用户可随时开关语音回复功能，进一步控制TTS调用成本。

6. 总结与展望

本项目成功实现了一个功能完整的AI视觉对话助手，用户可以通过摄像头和麦克风与AI进行自然的多模态交互。在12个规划的用户故事中，10个已完整实现，1个部分实现，实现率达83%。核心的视觉理解、语音交互和对话功能均已达到可用状态。

在成本控制方面，我们系统性地规划了12项成本控制技巧，最终采用了其中8项，形成了覆盖数据采集、传输、处理全链路的成本优化体系。通过端侧VAD、智能帧采样、图像压缩、场景变化检测和分级模型调用等核心策略，预计可降低60-75%的API调用成本，同时保持良好的用户体验。这些策略不仅适用于当前应用，也可为其他多模态AI应用的成本优化提供参考。

后续迭代方向包括：完善多语言支持以覆盖更广泛的用户群体；实现对话导出功能满足用户留存需求；引入WebSocket长连接提升高频交互场景的性能；探索端侧轻量模型的部署可能性，进一步降低云端依赖和运营成本。此外，还可以考虑增加多人视频通话场景的支持，以及与智能家居、在线教育等垂直领域的深度整合，拓展应用场景。